

系统开发工具实验报告 2

姓名：李博文 学号：25020007057

第 5 题控制一个可清理的后台任务

主题：命令行环境：进程、信号与任务控制 **建议用时：**12—15 分钟

1. 创建并进入 q05 目录

```
mkdir -p ~/q05  
cd ~/q05
```

2. 编辑 worker.sh，写入题目给定全部脚本内容

```
nano worker.sh
```

worker.sh 脚本内容：

```
#!/usr/bin/env bash  
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT  
n=0  
while true; do  
echo "$n"  
n=$((n+1))  
sleep 1  
done
```

3. 给脚本添加执行权限

```
chmod +x worker.sh
```

4. 前台启动脚本，标准输出、标准错误分别重定向日志文件

```
./worker.sh > stdout.log 2> stderr.log
```

此时终端被脚本占用，不会在屏幕打印数字，数字全部写到 stdout.log。

5. 按 Ctrl+Z 挂起当前前台任务
6. 使用 bg 命令，把挂起的任务放到后台继续运行
7. 用 jobs 查看后台任务，确认状态为 Running

```
jobs
```

8. 通过 `jobs -p` 获取任务 PID，发送 SIGTERM 信号，禁止 kill -9

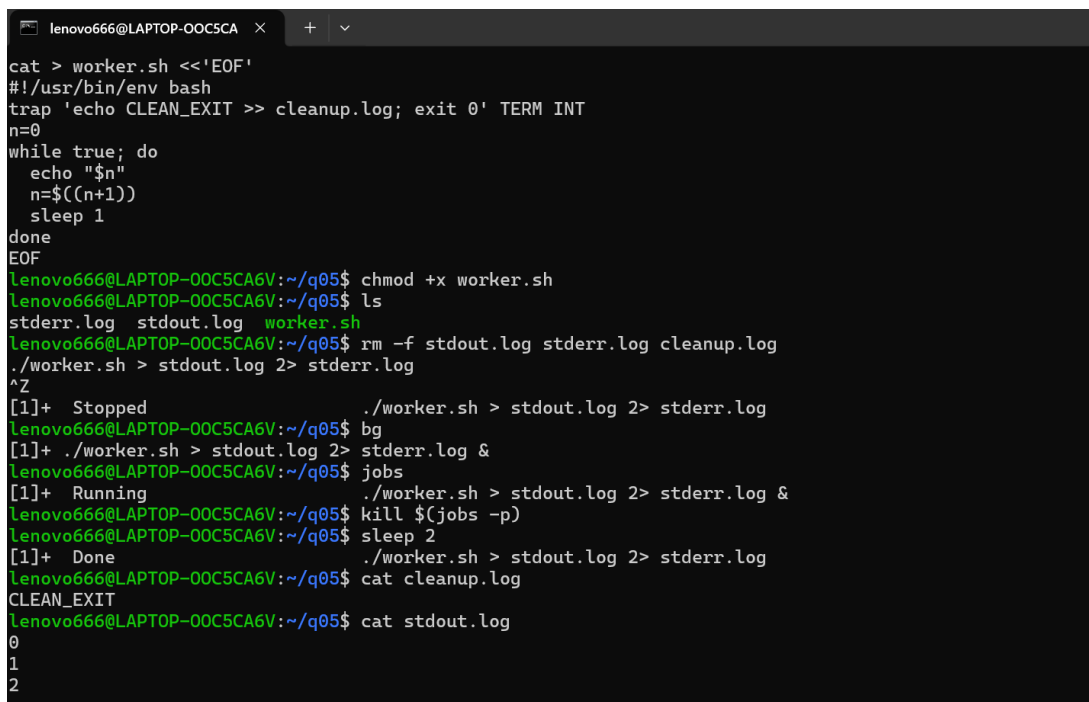
```
kill $(jobs -p)
```

9. 执行 `jobs` 确认任务已经退出，`jobs` 输出为空

```
jobs
```

10. 检查清理日志，验证是否输出 CLEAN_EXIT

```
cat cleanup.log
```



```
lenovo666@LAPTOP-00C5CA6V:~/q05$ cat > worker.sh <<'EOF'
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
EOF
lenovo666@LAPTOP-00C5CA6V:~/q05$ chmod +x worker.sh
lenovo666@LAPTOP-00C5CA6V:~/q05$ ls
stderr.log  stdout.log  worker.sh
lenovo666@LAPTOP-00C5CA6V:~/q05$ rm -f stdout.log stderr.log cleanup.log
lenovo666@LAPTOP-00C5CA6V:~/q05$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
lenovo666@LAPTOP-00C5CA6V:~/q05$ bg
[1]+ ./worker.sh > stdout.log 2> stderr.log &
lenovo666@LAPTOP-00C5CA6V:~/q05$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
lenovo666@LAPTOP-00C5CA6V:~/q05$ kill $(jobs -p)
lenovo666@LAPTOP-00C5CA6V:~/q05$ sleep 2
[1]+  Done                    ./worker.sh > stdout.log 2> stderr.log
lenovo666@LAPTOP-00C5CA6V:~/q05$ cat cleanup.log
CLEAN_EXIT
lenovo666@LAPTOP-00C5CA6V:~/q05$ cat stdout.log
0
1
2
```

图 1: q05 实验运行截图

第 6 题语义重构与本地开发反馈

主题：开发环境与工具 建议用时：12—15 分钟

1. 先安装 pipx

```
sudo apt update
sudo apt install -y pipx
```

2. 用 pipx 安装 ruff、pytest

```
pipx install ruff
pipx install pytest
```

3. 把 pipx 的程序路径加入 shell

```
pipx ensurepath
```

4. 创建三个 py 文件:

```
cat > math_utils.py <<'EOF'
def total_price(price: float, count: int)-> float:
    return price * count
EOF

cat > app.py <<'EOF'
from math_utils import total_price
print(total_price(12.5, 4))
EOF

cat > test_math_utils.py <<'EOF'
from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
EOF
```

5. 打开 Vs code, 连上 WSL Ubuntu

- 打开 app.py, 光标放到 `total_price`, 按 F12, 跳转到函数定义。

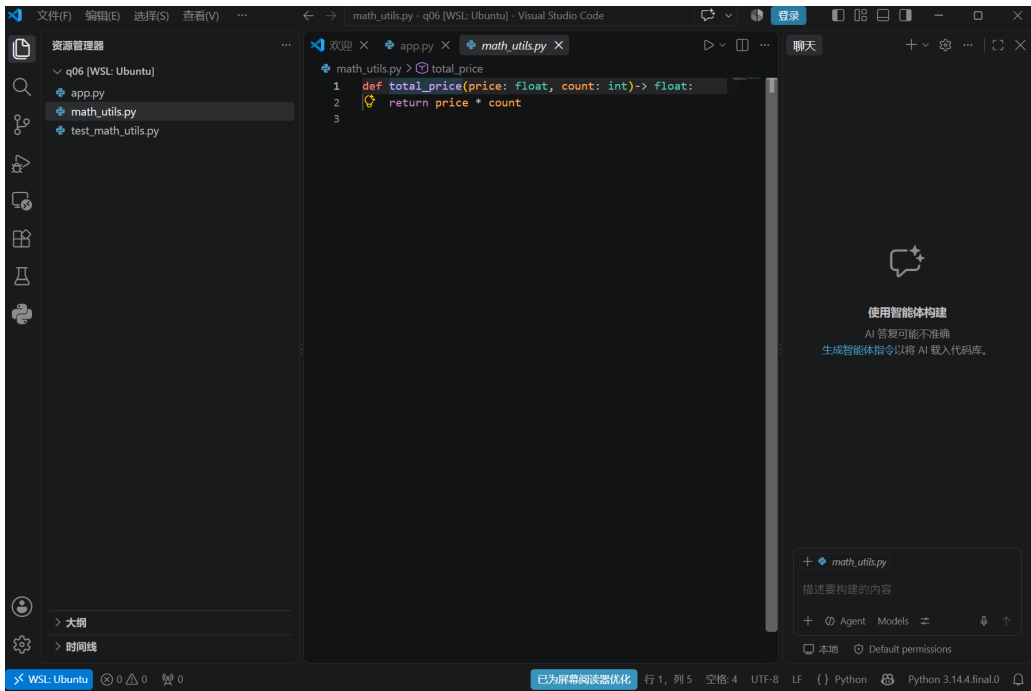


图 2: F12 跳转函数定义截图

- 光标放在函数名 `total_price`，按 `Shift+F12`，查看所有引用位置。

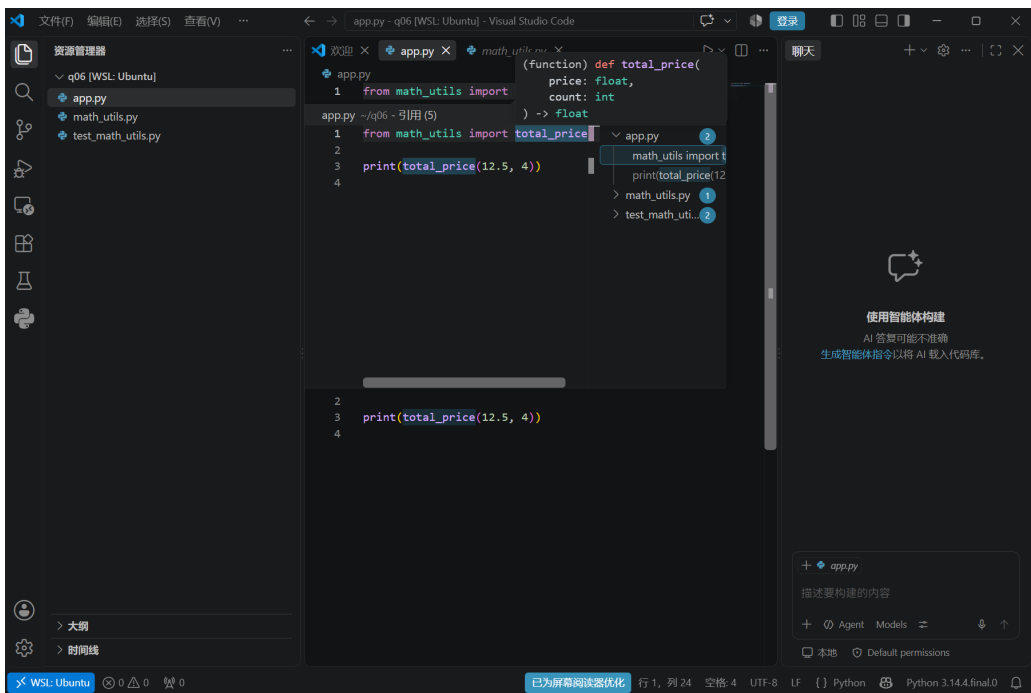


图 3: Shift+F12 查看全部引用截图

- 光标放在函数名，按 `F2`，输入新名字：`calculate_total`，回车。

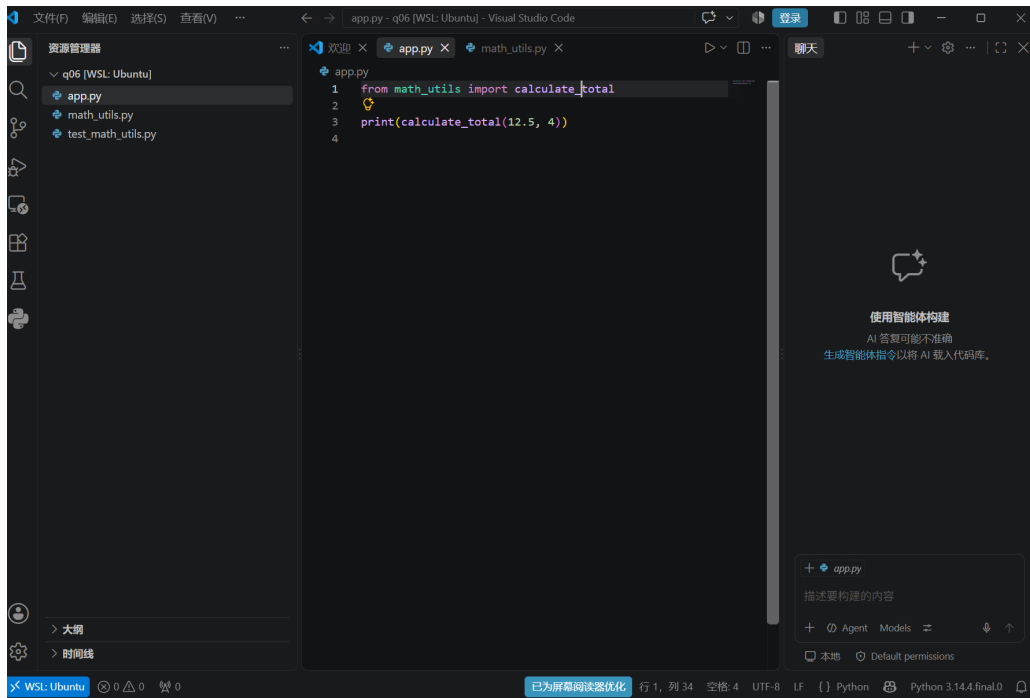


图 4: F2 语义重命名截图

6. 回到 WSL 终端，执行检查与测试

```
ruff check .
ruff check . --fix
pytest
```

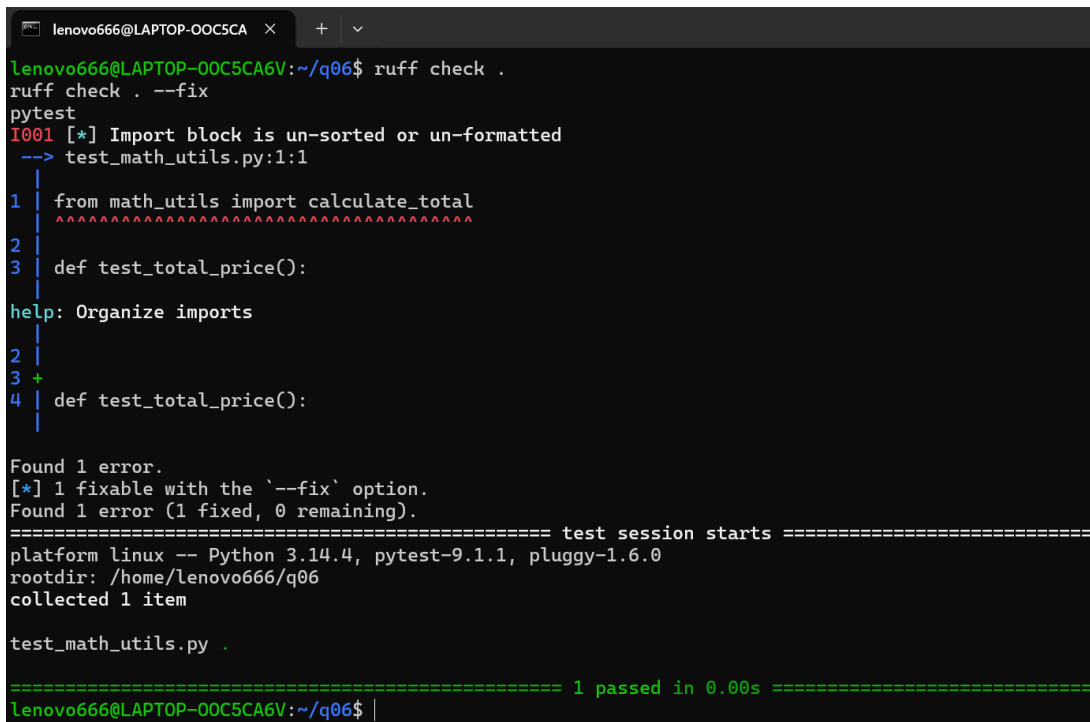


图 5: ruff 与 pytest 运行截图

第 7 题用调试器定位归并排序缺陷

主题: Debugging 建议用时: 12—15 分钟

1. 创建目录与文件

```
mkdir -p ~/q07
cd ~/q07
```

2. 新建 merge_sort.py, 复制下面带 bug 原始代码

```
cat > merge_sort.py <<'EOF'
def merge(left, right):
    import pdb; pdb.set_trace()
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
EOF
```

3. 运行原始有 bug 代码

```
python3 merge_sort.py
```

```
lenovo666@LAPTOP-OOC5CA  ×  +  ▾
-bash: return: a: numeric argument required
-bash: return: can only 'return' from a function or sourced script
-bash: syntax error near unexpected token '('
-bash: syntax error near unexpected token '('
-bash: syntax error near unexpected token 'merge_sort'
lenovo666@LAPTOP-OOC5CA6V:~/q07$ python3 merge_sort.py
python3: can't open file '/home/lenovo666/q07/merge_sort.py': [Errno 2] No such file or directory
lenovo666@LAPTOP-OOC5CA6V:~/q07$ cd ~/q07
lenovo666@LAPTOP-OOC5CA6V:~/q07$ cat > merge_sort.py <<'EOF'
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
EOF
lenovo666@LAPTOP-OOC5CA6V:~/q07$ python3 merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
lenovo666@LAPTOP-OOC5CA6V:~/q07$ |
```

图 6: bug 版本运行截图

4. pdb 分步调试

重复输入 n 多次, 直到执行进入 `while i < len(left) and j < len(right):` 这一行。

输入: `p i,j,left,right`。继续重复输入 n 多次, 当程序跑到 `else` 那一行: `result.append(right[j]); j += 1`, 输入 `p i,j`, 输入 `q` 退出调试。

```
lenovo666@LAPTOP-OOC5CA  ×  +  ▾
    return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
EOF
lenovo666@LAPTOP-OOC5CA6V:~/q07$ python3 merge_sort.py
> /home/lenovo666/q07/merge_sort.py(2)merge()
-> import pdb; pdb.set_trace()
(Pdb) n
> /home/lenovo666/q07/merge_sort.py(3)merge()
-> result = []
(Pdb) n
> /home/lenovo666/q07/merge_sort.py(4)merge()
-> i = j = 0
(Pdb) n
> /home/lenovo666/q07/merge_sort.py(5)merge()
-> while i < len(left) and j < len(right):
(Pdb) p i,j,left,right
(0, 0, [3], [1])
(Pdb) n
> /home/lenovo666/q07/merge_sort.py(6)merge()
-> if left[i] <= right[j]:
(Pdb) n
> /home/lenovo666/q07/merge_sort.py(9)merge()
-> result.append(right[j]); j += 1
(Pdb) p i,j
(0, 0)
(Pdb) q
Quitting pdb will kill the process. Quit anyway? [y/n] |
```

图 7: pdb 调试过程截图

5. 写入修复完成的代码（删掉 pdb 断点，right[i] 改成 right[j]）

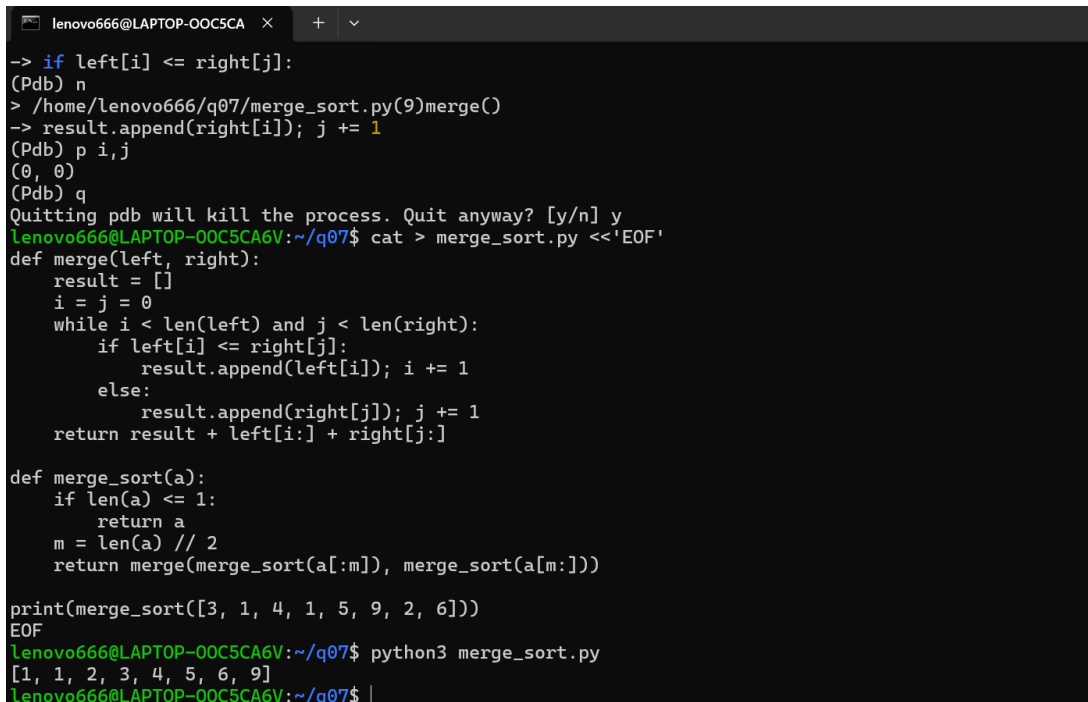
```
cat > merge_sort.py <<'EOF'
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
EOF
```

6. 运行修复后的程序

```
python3 merge_sort.py
```



```
lenovo666@LAPTOP-OOC5CA  x  +  v
-> if left[i] <= right[j]:
(Pdb) n
> /home/lenovo666/q07/merge_sort.py(9)merge()
-> result.append(right[i]); j += 1
(Pdb) p i,j
(0, 0)
(Pdb) q
Quitting pdb will kill the process. Quit anyway? [y/n] y
lenovo666@LAPTOP-OOC5CA6V:~/q07$ cat > merge_sort.py <<'EOF'
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
EOF
lenovo666@LAPTOP-OOC5CA6V:~/q07$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
lenovo666@LAPTOP-OOC5CA6V:~/q07$
```

图 8: 修复后运行截图

7. 创建 pytest 测试文件

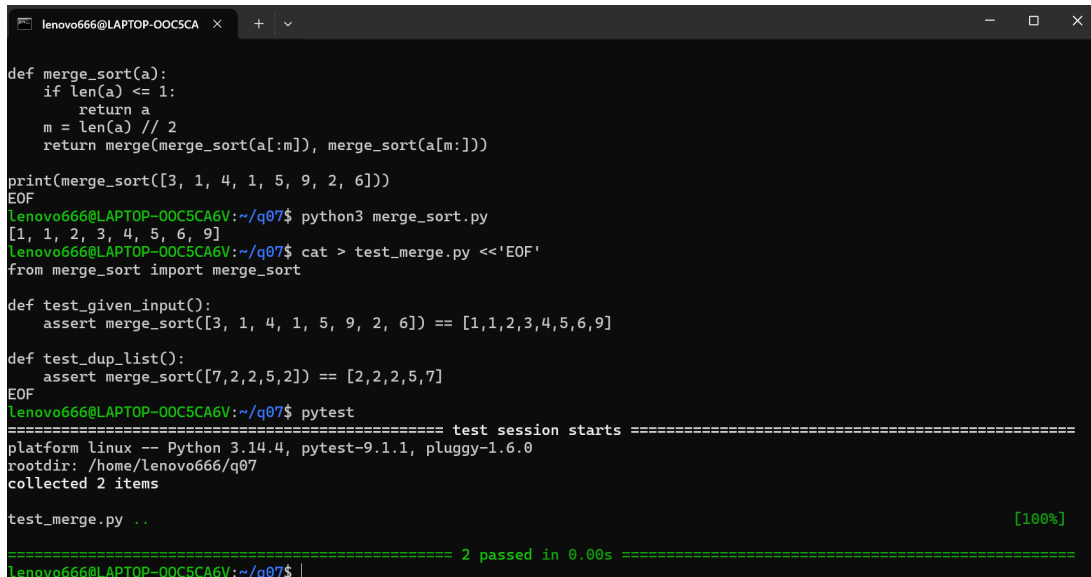
```
cat > test_merge.py <<'EOF'
from merge_sort import merge_sort

def test_given_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1,1,2,3,4,5,6,9]

def test_dup_list():
    assert merge_sort([7,2,2,5,2]) == [2,2,2,5,7]
EOF
```

8. 执行测试

```
pytest
```



```
lenovo666@LAPTOP-00C5CA  x  +  v  -  □  x
def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
EOF
lenovo666@LAPTOP-00C5CA6V:~/q07$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
lenovo666@LAPTOP-00C5CA6V:~/q07$ cat > test_merge.py <<'EOF'
from merge_sort import merge_sort

def test_given_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1,1,2,3,4,5,6,9]

def test_dup_list():
    assert merge_sort([7,2,2,5,2]) == [2,2,2,5,7]
EOF
lenovo666@LAPTOP-00C5CA6V:~/q07$ pytest
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/lenovo666/q07
collected 2 items

test_merge.py .. [100%]

===== 2 passed in 0.00s =====
lenovo666@LAPTOP-00C5CA6V:~/q07$
```

图 9: pytest 全部用例通过截图

第 8 题先测量，再优化慢速词频程序

主题: Profiling 建议用时: 12—15 分钟

1. 创建目录，生成文件

```
mkdir -p ~/q08
cd ~/q08
```

一. generate_words.py 生成词表

```
cat > generate_words.py <<'EOF'
import random
```

```

random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
EOF

```

运行生成 words.txt

```
python3 generate_words.py
```

二. 原始程序 wordfreq.py

```

cat > wordfreq.py <<'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
EOF

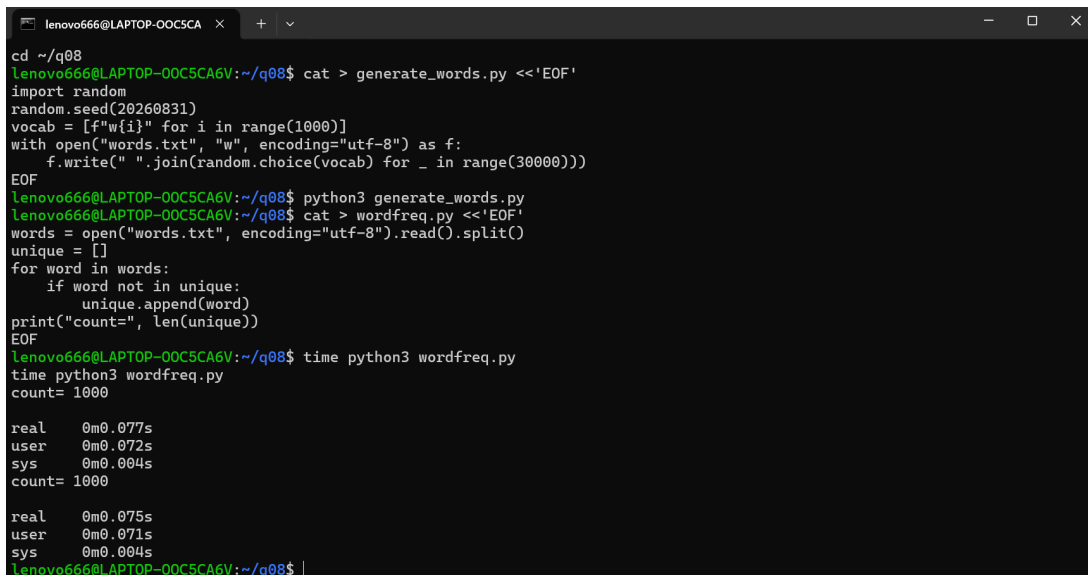
```

2. time 测耗时, 运行 2 次

```

time python3 wordfreq.py
time python3 wordfreq.py

```



```

lenovo666@LAPTOP-00C5CA6V:~/q08$ cat > generate_words.py <<'EOF'
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
EOF
lenovo666@LAPTOP-00C5CA6V:~/q08$ python3 generate_words.py
lenovo666@LAPTOP-00C5CA6V:~/q08$ cat > wordfreq.py <<'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
EOF
lenovo666@LAPTOP-00C5CA6V:~/q08$ time python3 wordfreq.py
time python3 wordfreq.py
count= 1000

real    0m0.077s
user    0m0.072s
sys     0m0.004s
count= 1000

real    0m0.075s
user    0m0.071s
sys     0m0.004s
lenovo666@LAPTOP-00C5CA6V:~/q08$ |

```

图 10: 原始版本 time 计时截图

3. cProfile 找耗时点

```
python3 -m cProfile -s cumulative wordfreq.py
```

```
lenovo666@LAPTOP-OOC5CA x + v
user 0m0.072s
sys 0m0.004s
count= 1000

real 0m0.075s
user 0m0.071s
sys 0m0.004s
lenovo666@LAPTOP-OOC5CA6V:~/q08$ python3 -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.068 seconds

Ordered by: cumulative time

ncalls tottime percall cumtime percall filename:lineno(function)
1 0.000 0.000 0.068 0.068 {built-in method builtins.exec}
1 0.067 0.067 0.068 0.068 wordfreq.py:1(<module>)
1 0.001 0.001 0.001 0.001 {method 'split' of 'str' objects}
1 0.000 0.000 0.000 0.000 {method 'read' of 'io.TextIOWrapper' objects}
1 0.000 0.000 0.000 0.000 <frozen codecs>:322(decode)
1 0.000 0.000 0.000 0.000 {built-in method _codecs.utf_8_decode}
1000 0.000 0.000 0.000 0.000 {method 'append' of 'list' objects}
1 0.000 0.000 0.000 0.000 {built-in method builtins.print}
1 0.000 0.000 0.000 0.000 {method 'disable' of '_lsprof.Profiler' objects}
1 0.000 0.000 0.000 0.000 {built-in method _io.open}
1 0.000 0.000 0.000 0.000 <frozen codecs>:312(__init__)
1 0.000 0.000 0.000 0.000 {built-in method builtins.len}
1 0.000 0.000 0.000 0.000 <frozen codecs>:263(__init__)

lenovo666@LAPTOP-OOC5CA6V:~/q08$
```

图 11: cProfile 性能分析截图

4. 优化版本 wordfreq_opt.py

```
cat > wordfreq_opt.py <<'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique_set = set()
for word in words:
    unique_set.add(word)
unique = list(unique_set)
print("count=", len(unique))
EOF
```

5. 运行 2 次计时

```
time python3 wordfreq_opt.py
time python3 wordfreq_opt.py
```

```
lenovo666@LAPTOP-00C5CA X + v
1000 0.000 0.000 0.000 0.000 {method 'append' of 'list' objects}
1 0.000 0.000 0.000 0.000 {built-in method builtins.print}
1 0.000 0.000 0.000 0.000 {method 'disable' of '_lsprof.Profiler' objects}
1 0.000 0.000 0.000 0.000 {built-in method _io.open}
1 0.000 0.000 0.000 0.000 <frozen codecs>:312(__init__)
1 0.000 0.000 0.000 0.000 {built-in method builtins.len}
1 0.000 0.000 0.000 0.000 <frozen codecs>:263(__init__)

lenovo666@LAPTOP-00C5CA6V:~/q08$ cat > wordfreq_opt.py <<'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique_set = set()
for word in words:
    unique_set.add(word)
unique = list(unique_set)
print("count=", len(unique))
EOF
lenovo666@LAPTOP-00C5CA6V:~/q08$ time python3 wordfreq_opt.py
time python3 wordfreq_opt.py
count= 1000

real    0m0.009s
user    0m0.009s
sys     0m0.000s
count= 1000

real    0m0.009s
user    0m0.004s
sys     0m0.005s
lenovo666@LAPTOP-00C5CA6V:~/q08$
```

图 12: 优化后版本计时截图

练习

练习 1

题目：你可能见过像 `cmd --flag -- --notafg` 这样的命令。这里的 `--` 是一个特殊参数，它告诉程序后面不要再继续解析选项 (flag) 了。也就是说，`--` 后面的所有内容都会被当作位置参数 (positional argument)。这有什么用？试着运行 `touch -- -myfile`，然后在不使用 `--` 的情况下把它删掉。

答：`--` 是命令行选项终止符，通知程序停止解析后续以 `-` 开头的内容为命令选项 (flag)，`--` 之后全部内容视作普通位置参数。

```
lenovo666@LAPTOP-00C5CA X + v
Microsoft Windows [版本 10.0.26290.9168]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\Lenovo>wsl ~
lenovo666@LAPTOP-00C5CA6V:~$ touch -- -myfile
lenovo666@LAPTOP-00C5CA6V:~$ rm ./-myfile
lenovo666@LAPTOP-00C5CA6V:~$
```

图 13: 练习 1 运行截图

练习 2

题目：进程替换 `<(command)` 可以让你把一个命令的输出当成文件来用。试着配合 `diff` 和进程替换，比较 `printenv` 与 `export` 的输出。它们为什么不一样？（提示：可以试试 `diff <(printenv | sort) <(export | sort)`）

答：

```
diff <(printenv | sort) <(export | sort)
```

1. 输出格式不同。`export` 输出是 shell 声明语句：`declare -x HOME="/home/xxx"; printenv` 只输出变量 = 值的原始内容，没有 `declare -x` 前缀。即使变量集合相同，文本行不一样，`diff` 会报很多行差异。

2. 变量集合不完全等同。`printenv`：全部已经导出生效的环境变量，会包含继承来的环境变量。`export` 只列出 shell 里被标记为要导出的变量；有些 shell 内部变量不一定出现在两者中。被 `export` 标记的变量才会进入 `printenv` 的环境；但二者输出文本格式完全不同，排序后 `diff` 仍然看到大量不同。

练习 3

题目：阅读 `man ls`，写出一个 `ls` 命令，让它按下面的方式列出文件：

- 包含所有文件，包括隐藏文件
- 文件大小以易读格式显示（例如 454M，而不是 454279954）
- 按最近修改时间排序
- 输出带颜色

示例输出大概像这样：

```
-rw-r--r--  1 user group 1.1M Jan 14 09:53 baz
drwxr-xr-x  5 user group  160 Jan 14 09:53 .
-rw-r--r--  1 user group  514 Jan 14 06:42 bar
-rw-r--r--  1 user group 106M Jan 13 12:12 foo
drwx-----+ 47 user group 1.5K Jan 12 18:08 ..
```

答：

```
ls -lath --color
```

```
lenovo666@LAPTOP-OOC5CA6V:~$ ls -lath --color
total 156K
drwxr-xr-x 17 lenovo666 lenovo666 4.0K Sep  7 14:05 .
-rw-rw-r-- 1 lenovo666 lenovo666   0 Sep  7 13:57 .motd_shown
-rw----- 1 lenovo666 lenovo666 33K Sep  5 16:40 .bash_history
drwxr-xr-x  2 lenovo666 lenovo666 4.0K Sep  5 16:35 q08
drwxr-xr-x  4 lenovo666 lenovo666 4.0K Sep  5 16:31 q07
drwxr-xr-x  5 lenovo666 lenovo666 4.0K Sep  5 15:59 q06
drwxr-xr-x  3 lenovo666 lenovo666 4.0K Sep  5 15:46 .dotnet
drwx----- 3 lenovo666 lenovo666 4.0K Sep  5 15:46 .copilot
drwxr-xr-x  7 lenovo666 lenovo666 4.0K Sep  5 15:46 .cache
drwxr-xr-x  5 lenovo666 lenovo666 4.0K Sep  5 15:46 .vscode-server
-rw-r--r--  1 lenovo666 lenovo666 898 Sep  5 15:17 .profile
-rw-r--r--  1 lenovo666 lenovo666 3.8K Sep  5 15:17 .bashrc
drwxr-xr-x  5 lenovo666 lenovo666 4.0K Sep  5 15:16 .local
drwxr-xr-x  2 lenovo666 lenovo666 4.0K Sep  5 14:56 q05
drwxr-xr-x  3 lenovo666 lenovo666 4.0K Sep  1 22:59 lab_report
-rw-----  1 lenovo666 lenovo666 20 Sep  1 18:58 .lessht
-rw-r--r--  1 lenovo666 lenovo666 52 Sep  1 18:58 .gitconfig
drwxr-xr-x  2 lenovo666 lenovo666 4.0K Sep  1 18:51 q04
-rw-r--r--  1 lenovo666 lenovo666   0 Sep  1 17:39 real.txt
-rwxr-xr-x  1 lenovo666 lenovo666 184 Sep  1 17:39 check.sh
-rwxr-xr-x  1 lenovo666 lenovo666 128 Aug 31 15:59 trace.sh
-rwxr-xr-x  1 lenovo666 lenovo666 116 Aug 31 15:38 checkfile.sh
-rw-r--r--  1 lenovo666 lenovo666 233 Aug 31 15:33 all.txt
-rw-r--r--  1 lenovo666 lenovo666  60 Aug 31 15:33 err.txt
-rw-r--r--  1 lenovo666 lenovo666 173 Aug 31 15:33 out.txt
-rw-r--r--  1 lenovo666 lenovo666 5.5K Aug 30 22:57 commands.txt
drwxr-xr-x  3 lenovo666 lenovo666 4.0K Aug 30 22:06 q03
drwxr-xr-x  2 lenovo666 lenovo666 4.0K Aug 30 21:44 q02
```

图 14: 练习 3 ls 命令截图

练习 4

题目：写两个 bash 函数 marco 和 polo，行为如下：每次执行 marco 时，都要以某种方式保存当前工作目录；之后无论你切到哪个目录，只要执行 polo，它都应该把你 cd 回执行 marco 时所在的目录。为了方便调试，你可以把代码写进 marco.sh，然后执行 source marco.sh 把这些定义重新加载到当前 shell。

答： marco.sh 代码：

```
marco() {
    MARCO_SAVED_DIR="$PWD"
}

polo() {
    cd "$MARCO_SAVED_DIR"
}
```

加载脚本到当前 shell：

```
source marco.sh
```

```
lenovo666@LAPTOP-00C5CA  x  +  v  -  □  x
-rw-r--r--  1 lenovo666 lenovo666 233 Aug 31 15:33 all.txt
-rw-r--r--  1 lenovo666 lenovo666  60 Aug 31 15:33 err.txt
-rw-r--r--  1 lenovo666 lenovo666 173 Aug 31 15:33 out.txt
-rw-r--r--  1 lenovo666 lenovo666 5.5K Aug 30 22:57 commands.txt
drwxr-xr-x  3 lenovo666 lenovo666 4.0K Aug 30 22:06 q03
drwxr-xr-x  2 lenovo666 lenovo666 4.0K Aug 30 21:44 q02
drwxr-xr-x  4 lenovo666 lenovo666 4.0K Aug 30 21:29 q01
drwxr-xr-x  3 lenovo666 lenovo666 4.0K Aug 30 20:56 .config
-rw-r--r--  1 lenovo666 lenovo666 220 Aug 30 20:51 .bash_logout
drwxr-xr-x  3 root      root      4.0K Aug 30 20:51 ..
lenovo666@LAPTOP-00C5CA6V:~$ cat > marco.sh <<'EOF'
marco() {
    MARCO_SAVED_DIR="$PWD"
}

polo() {
    cd "$MARCO_SAVED_DIR"
}
EOF
lenovo666@LAPTOP-00C5CA6V:~$ source marco.sh
lenovo666@LAPTOP-00C5CA6V:~$ pwd
/home/lenovo666
lenovo666@LAPTOP-00C5CA6V:~$ marco
lenovo666@LAPTOP-00C5CA6V:~$ cd /tmp
lenovo666@LAPTOP-00C5CA6V:/tmp$ pwd
/tmp
lenovo666@LAPTOP-00C5CA6V:/tmp$ polo
lenovo666@LAPTOP-00C5CA6V:~$ pwd
/home/lenovo666
lenovo666@LAPTOP-00C5CA6V:~$ |
```

图 15: 练习 4 marco polo 截图

练习 5

题目: 创建一个 alias dc, 让你把 cd 打错时也能正常工作。

答:

```
alias dc='cd'
```

```
lenovo666@LAPTOP-00C5CA6V:~$ alias dc='cd'
lenovo666@LAPTOP-00C5CA6V:~$ pwd
dc /tmp
pwd
dc ~
pwd
/home/lenovo666
/tmp
/home/lenovo666
lenovo666@LAPTOP-00C5CA6V:~$ |
```

图 16: 练习 5 alias dc 截图

练习 6

题目: 运行 `history | awk '$1="" ; print substr($0,2)' | sort | uniq -c | sort -n | tail -n 10`, 找出你最常用的 10 条命令, 并考虑给它们写更短的 alias。注意: 这条命令适用于 Bash; 如果你用的是 ZSH, 把 `history` 换成 `history 1`。

答:

```
history | awk '{\ $1="" ; print substr(\ $0,2)}' | sort | uniq -c | sort -n |
tail -n 10
```

> 提示 zsh 用户将 `history` 替换为 `history 1`。

```
lenovo666@LAPTOP-00C5CA6V:~$ history | awk '{s1="";print substr($0,2)}' | sort | uniq -c | sort -n | tail -n 10
  10 latexmk -C
  11 #!/bin/bash
  13 cd ~/lab_report
  14 pwd
  16 \hline
  26 \begin{lstlisting}
  26 \end{figure}
  26 \end{lstlisting}
  28 EOF
  30 \centering
lenovo666@LAPTOP-00C5CA6V:~$ alias lab='cd ~/lab_report'
alias clatex='latexmk -C'
lenovo666@LAPTOP-00C5CA6V:~$ |
```

图 17: 练习 6 历史命令统计截图

练习 7

题目：调试排序算法：以下伪代码实现了合并排序，但存在一个错误。用你选择的语言实现它，然后用调试器（gdb、lldb、pdb，或你 IDE 的调试器）来查找并修复 bug。

```
function merge_sort(arr):
    if length(arr) <= 1:
        return arr
    mid = length(arr) / 2
    left = merge_sort(arr[0..mid])
    right = merge_sort(arr[mid..end])
    return merge(left, right)

function merge(left, right):
    result = []
    i = 0, j = 0
    while i < length(left) AND j < length(right):
        if left[i] <= right[j]:
            append result, left[i]
            i = i + 1
        else:
            append result, right[i]
            j = j + 1
    append remaining elements from left and right
    return result
```

测试向量：merge_sort([3, 1, 4, 1, 5, 9, 2, 6])，应返回 [1, 1, 2, 3, 4, 5, 6, 9]。
使用断点并逐步通过合并函数找出错误元素被选中的位置。

答：错误位置：merge 函数内 else 分支写 result.append(right[i]); j +=1，下标错误，应该是 right[j]。

```
lenovo666@LAPTOP-OOC5CA X + v
(Pdb) n
> /home/lenovo666/merge_sort.py(17)merge()
-> if left[i] <= right[j]:
(Pdb) n
> /home/lenovo666/merge_sort.py(21)merge()
-> result.append(right[i])
(Pdb) p i
0
(Pdb) p j
0
(Pdb) p left
[3]
(Pdb) p right
[1]
(Pdb) n
> /home/lenovo666/merge_sort.py(22)merge()
-> j = j + 1
(Pdb) p result
[1]
(Pdb) n
> /home/lenovo666/merge_sort.py(16)merge()
-> while i < len(left) and j < len(right):
(Pdb) p j
1
(Pdb) p result
[1]
(Pdb) c
> /home/lenovo666/merge_sort.py(12)merge()
-> pdb.set_trace()
(Pdb) |
```

图 18: 练习 7pdb 调试截图 1

```
result.extend(left[i:])
result.extend(right[j:])
return result

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
EOF
lenovo666@LAPTOP-OOC5CA6V:~$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
lenovo666@LAPTOP-OOC5CA6V:~$ |
```

图 19: 练习 7pdb 调试截图 2

练习 8

题目：使用 AddressSanitizer 调试内存错误。保存为：uaf.c

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

int main() {
    char *greeting = malloc(32);
    strcpy(greeting, "Hello, world!");
    printf("%s\n", greeting);

    free(greeting);

    greeting[0] = 'J';
    printf("%s\n", greeting);

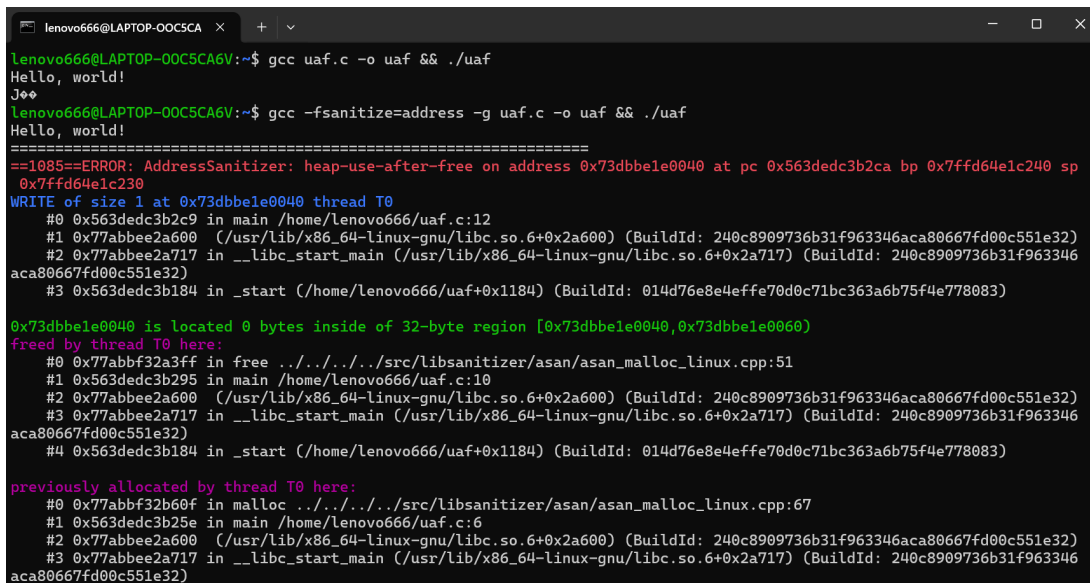
    return 0;
}
```

首先编译并运行时不使用消毒剂: `gcc uaf.c -o uaf && ./uaf`。看起来它有效。现在用 AddressSanitizer 编译: `gcc -fsanitize=address -g uaf.c -o uaf && ./uaf`。阅读错误报告。ASan 发现了什么 bug? 解决它识别出的问题。

答: 编译运行命令:

```
gcc uaf.c -o uaf && ./uaf
gcc -fsanitize=address -g uaf.c -o uaf && ./uaf
```

ASan 检测出 **heap-use-after-free (释放后继续访问堆内存)**。内存 free 之后不允许读写这块内存。修复: 调整 free 放到读写之后。



```
lenovo666@LAPTOP-OOC5CA6V:~$ gcc uaf.c -o uaf && ./uaf
Hello, world!
J00
lenovo666@LAPTOP-OOC5CA6V:~$ gcc -fsanitize=address -g uaf.c -o uaf && ./uaf
Hello, world!
=====
==1085==ERROR: AddressSanitizer: heap-use-after-free on address 0x73dbb1e0040 at pc 0x563dedc3b2ca bp 0x7ffd64e1c240 sp
0x7ffd64e1c230
WRITE of size 1 at 0x73dbb1e0040 thread T0
#0 0x563dedc3b2c9 in main /home/lenovo666/uaf.c:12
#1 0x77abbee2a600 (/usr/lib/x86_64-linux-gnu/libc.so.6+0x2a600) (BuildId: 240c8909736b31f963346aca80667fd00c551e32)
#2 0x77abbee2a717 in __libc_start_main (/usr/lib/x86_64-linux-gnu/libc.so.6+0x2a717) (BuildId: 240c8909736b31f963346
aca80667fd00c551e32)
#3 0x563dedc3b184 in _start (/home/lenovo666/uaf+0x1184) (BuildId: 014d76e8e4effe70d0c71bc363a6b75f4e778083)

0x73dbb1e0040 is located 0 bytes inside of 32-byte region [0x73dbb1e0040,0x73dbb1e0060)
freed by thread T0 here:
#0 0x77abbf32a3ff in free ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:51
#1 0x563dedc3b295 in main /home/lenovo666/uaf.c:10
#2 0x77abbee2a600 (/usr/lib/x86_64-linux-gnu/libc.so.6+0x2a600) (BuildId: 240c8909736b31f963346aca80667fd00c551e32)
#3 0x77abbee2a717 in __libc_start_main (/usr/lib/x86_64-linux-gnu/libc.so.6+0x2a717) (BuildId: 240c8909736b31f963346
aca80667fd00c551e32)
#4 0x563dedc3b184 in _start (/home/lenovo666/uaf+0x1184) (BuildId: 014d76e8e4effe70d0c71bc363a6b75f4e778083)

previously allocated by thread T0 here:
#0 0x77abbf32b60f in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:67
#1 0x563dedc3b25e in main /home/lenovo666/uaf.c:6
#2 0x77abbee2a600 (/usr/lib/x86_64-linux-gnu/libc.so.6+0x2a600) (BuildId: 240c8909736b31f963346aca80667fd00c551e32)
#3 0x77abbee2a717 in __libc_start_main (/usr/lib/x86_64-linux-gnu/libc.so.6+0x2a717) (BuildId: 240c8909736b31f963346
aca80667fd00c551e32)
```

图 20: 练习 8 ASan 报错截图

```
lenovo666@LAPTOP-OOC5CA  x  +  v
Poisoned by user:          f7
Container overflow:        fc
Array cookie:              ac
Intra object redzone:      bb
ASan internal:             fe
Left alloca redzone:       ca
Right alloca redzone:      cb
==1085==ABORTING
lenovo666@LAPTOP-OOC5CA6V:~$ cat > uaf.c <<'EOF'
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

int main() {
    char *greeting = malloc(32);
    strcpy(greeting, "Hello, world!");
    printf("%s\n", greeting);

    greeting[0] = 'J';
    printf("%s\n", greeting);

    free(greeting);

    return 0;
}
EOF
lenovo666@LAPTOP-OOC5CA6V:~$ gcc -fsanitize=address -g uaf.c -o uaf && ./uaf
Hello, world!
Jello, world!
lenovo666@LAPTOP-OOC5CA6V:~$ |
```

图 21: 练习 8 修复代码运行截图

练习 9

题目：用来获取你选择的程序的基本性能统计数据。不同的计数器是什么意思？

答：

- `cycles`: CPU 时钟周期；`instructions` 执行指令，二者比值 IPC 衡量 CPU 流水线效率。
- `branches` / `branch-misses` 统计分支指令与预测失败，分支失败会造成性能损耗。
- `task-clock` 程序占用 CPU 时间；`context-switches` 上下文切换；`cpu-migrations` 进程跨核心迁移；`page-faults` 虚拟内存缺页。
- `elapsed` 是程序真实总运行时间。

练习 10

题目：用于在运行资源密集型程序时监控系统。尝试用 `htop`、`taskset` 来限制进程可以使用的 CPU: `taskset --cpu-list 0,2 stress -c 3`。为什么不使用三颗 CPU？

答：命令：

```
taskset --cpu-list 0,2 stress -c 3
```

原因：`taskset --cpu-list` 用来限制进程允许调度的 CPU 核心；`stress -c N` 产生 N 个 CPU 压力线程。命令 `taskset --cpu-list 0,2 stress -c 3`: CPU 列表只提供核心 0、2，仅 2 颗

核心，却启动 3 个 CPU 密集线程。因此无法使用 3 颗 CPU；3 个线程竞争 2 个 CPU，会出现大量上下文切换，CPU1 始终空闲。如果要使用 3 颗 CPU，cpulist 应当写 3 个不同核心编号，例如 0,1,2。